

Bangladesh University of Engineering and Technology (BUET)

Department of Computer Science and Engineering

Final Report

Tool 9: Dictionary Attack and Known Password Attack

Course: CSE406: Computer Security Sessional

Assignment: Attack Tool 9

Report: Final Report and Implementation Demo

Group Members

Member	Student ID	Responsibility
Arnob Biswas	2105015	Webapp + defense mechanism
Zaki Rehnoom Unmona	2105016	Attacker tools + report diagrams

September 18, 2026

Contents

1	Overview of the Project	1
2	Network Topology, Components, and Timing Diagrams	1
2.1	Topology	1
2.2	Components	2
2.3	Timing Diagrams	3
2.3.1	Normal (legitimate) signup and login: browser flow	3
2.3.2	Dictionary attack: repeated exchange, one account	4
2.3.3	Known-password (spray) attack: repeated exchange, many accounts	4
2.3.4	With the lockout defense active	5
3	Packet and Frame Structure	5
3.1	Request (attacker or browser to server), one per login attempt	5
3.2	Responses, by outcome	6
3.3	Design Notes	6
4	Test Environment	6
5	Steps of the Attacks	7
5.1	Victim action: a normal signup and login	7
5.2	Dictionary attack, step by step	7
5.3	Known password (spray) attack, step by step	8
6	Observations	8
6.1	Dictionary attack, undefended	8
6.2	Dictionary attack, defended	9
6.3	Known password attack, undefended	10
6.4	Known password attack, defended	10
7	Assumptions	11
8	Defense Mechanism: Design and Assessment	12
9	Conclusion	13

1 Overview of the Project

We built and demonstrated two related password guessing attacks against a signup and login webapp that we wrote ourselves, along with a working defense against both.

- **Dictionary attack.** We target one specific account and try a long, ordered list of candidate passwords against it, one at a time, until the server accepts one or the list runs out. This works because many people choose passwords that show up in common wordlists.
- **Known password attack** (password spraying, or credential reuse). Instead of guessing many passwords against one account, we start from a small set of known passwords, such as defaults or passwords leaked in earlier breaches, and try each one against a wide list of accounts. This exploits password reuse and default credentials rather than weak individual passwords.

The target is a webapp we built from scratch rather than an existing service, a small signup and login application written in Flask with a SQLite database. The victim action is a normal user signing up and logging in through a browser. The attacker action is the two attacks above, driven by HTTP client code we wrote ourselves, not an existing brute forcing tool. Alongside the attacks, we built a defense mechanism, account lockout combined with slow password hashing, so that this report can show the attacks succeeding and then being stopped once the defense is switched on.

2 Network Topology, Components, and Timing Diagrams

2.1 Topology

We ran the attack on two ordinary devices connected to the same local network, with no virtual machines involved. The victim device runs the webapp and is reachable at 192.168.68.107 on port 5000. The attacker device sits on the same network and talks to the victim over plain HTTP. A browser on the same network is used for the legitimate signup and login action.

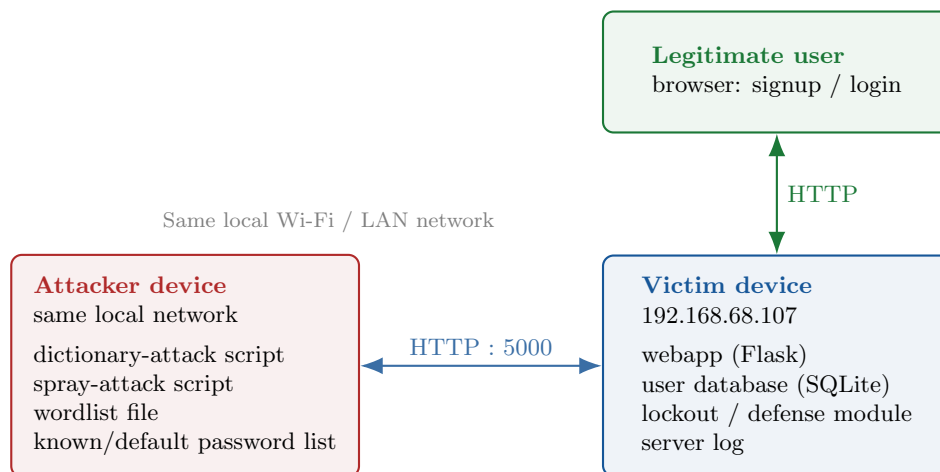


Figure 1: Attack topology: two devices on the same local network, plus a legitimate browser client on the same network exercising the webapp. No virtual machines are used.

2.2 Components

Component	Role
Victim webapp (Flask)	Serves signup, login, and dashboard pages, the attack target
User database (SQLite)	Stores usernames, salts, and password hashes
Defense module	Tracks failed login attempts per account and enforces temporary lockout, also selects the slow-hash mode
Dictionary-attack script	Attacker tool: many candidate passwords against one account
Spray-attack script	Attacker tool: a few known passwords against many accounts
Wordlist / known-password list	Input data for the two attacker scripts
Server / attacker logs	Record every login attempt and outcome

Table 1: System components and their roles.

2.3 Timing Diagrams

The diagrams below describe the protocol our webapp implements and the attack strategies we ran against it. Request arrows (client to server) are shown in **steel blue**. Responses are colored by outcome: **green** for **success**, **amber** for **a failed attempt**, and **red** for **a logout**.

2.3.1 Normal (legitimate) signup and login: browser flow

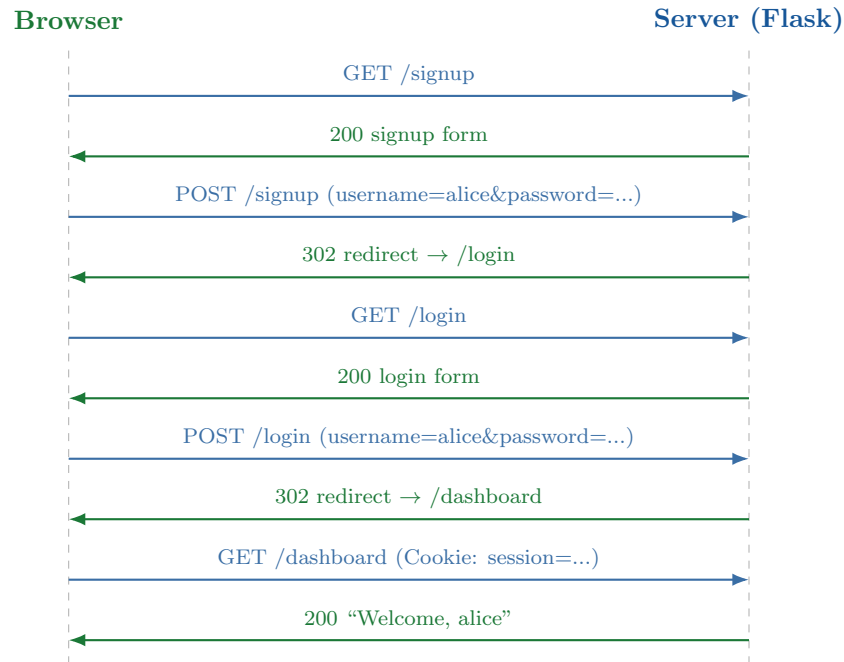


Figure 2: Normal protocol timing: one legitimate signup followed by one login.

2.3.2 Dictionary attack: repeated exchange, one account

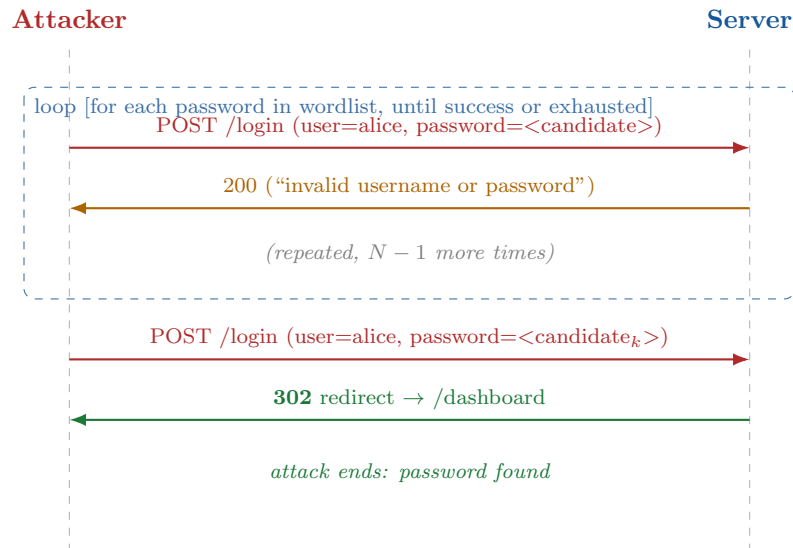


Figure 3: Dictionary attack: many candidate passwords tried in sequence against a single account.

2.3.3 Known-password (spray) attack: repeated exchange, many accounts

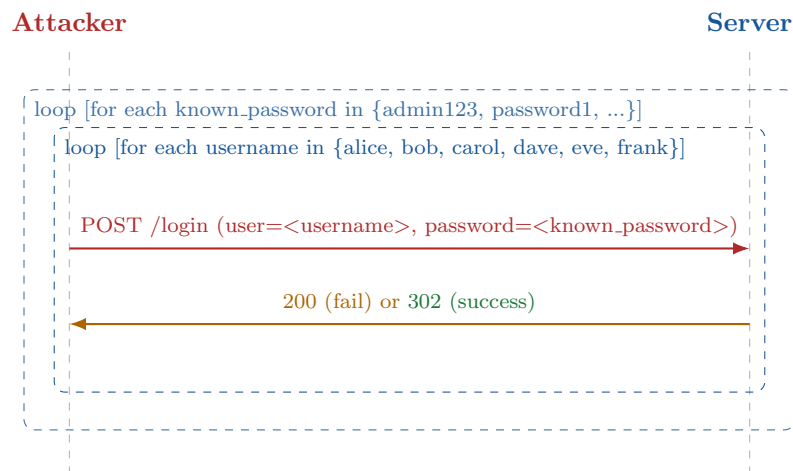


Figure 4: Known-password / spray attack: few known passwords tried across many accounts.

2.3.4 With the lockout defense active

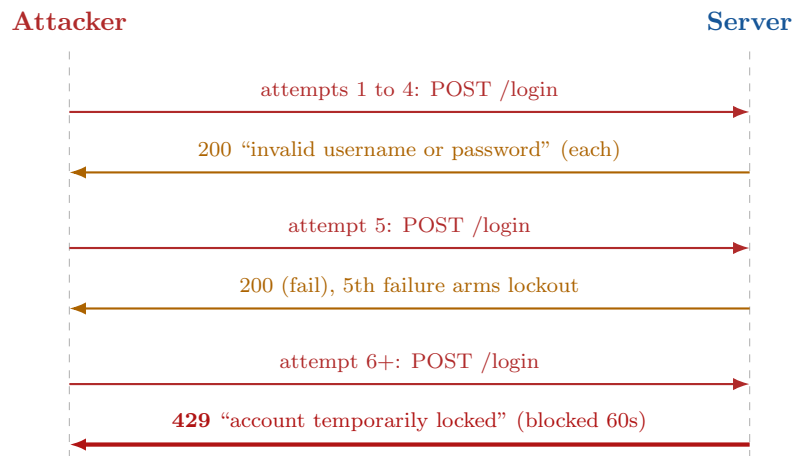


Figure 5: Lockout defense: the dictionary and spray attacks are blocked after 5 failed attempts within a 30-second window.

3 Packet and Frame Structure

No raw Ethernet or IP crafting is needed for this attack, since the exploit sits entirely at the application layer, carried over an ordinary TCP connection. The transport is standard TCP carrying HTTP/1.1, using Flask’s built-in development server. What we designed from scratch is the application behaviour and the payload the HTTP request carries, and how the server interprets it, not an existing authentication protocol implementation.

3.1 Request (attacker or browser to server), one per login attempt

```
POST /login HTTP/1.1
Host: 192.168.68.107:5000
Content-Type: application/x-www-form-urlencoded
Content-Length: 33

username=alice&password=sunshine
```

3.2 Responses, by outcome

Outcome	Status	Notable headers / body
Success	302 FOUND	Location: /dashboard, Set-Cookie: session=...
Failure	200 OK	login page HTML containing “invalid username or password”
Locked (defense on)	429 TOO MANY REQUESTS	login page HTML containing “account temporarily locked”

Table 2: HTTP response codes and bodies for each login outcome.

3.3 Design Notes

- Each attempt opens its own TCP connection, since the attacker scripts issue one HTTP request per attempt with no session reuse: a full three-way handshake, the HTTP request and response, then a FIN/ACK teardown, for every attempt. We captured this on the wire between the two devices with Wireshark, shown below.
- Whether the username does not exist or the password is simply wrong, the response to a failed login is the same generic message, “invalid username or password”, so the app never leaks whether an account exists.
- Passwords are sent as cleartext form data over plain HTTP, with no TLS. This is a deliberate simplification, consistent with the assignment’s other plaintext-protocol attacks, and it is worth noting as a limitation compared to a production deployment, which would use HTTPS.

4 Test Environment

The results in this report come from an actual two-device setup on the same local network, not from virtual machines and not from a single machine over loopback.

- **Victim device.** Runs the Flask webapp bound to all interfaces, reachable at `http://192.168.68.107:5000` from the rest of the network. Started undefended with `python seed.py` followed by `python app.py`, which hashes passwords with a single fast salted SHA-256 and enforces no lockout. Started defended with a fresh `app.db`, `python seed.py --defense`, and `DEFENSE=1 python app.py`, which hashes with salted PBKDF2-HMAC-SHA256 at 200,000 iterations and turns the lockout tracker on.
- **Attacker device.** On the same local network as the victim, pointed at `http://192.168.68.107:5000`. Runs `attacker_dict.py` against the account `alice` using `wordlist.txt` (1000 candidate passwords), and `attacker_spray.py` against the six accounts in `targets.txt` using the 33 passwords in `known_passwords.txt`.

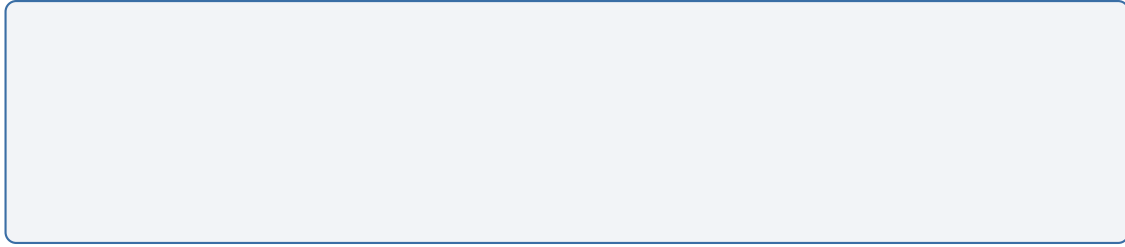


Figure 6: Network interface output from the victim device and the attacker device, showing both are on the 192.168.68.0/24 network.

5 Steps of the Attacks

5.1 Victim action: a normal signup and login

Before running either attack, we exercised the webapp the way a normal user would, to confirm the signup and login flow works on its own.

1. Open `http://192.168.68.107:5000/signup` in a browser and create a new account.
2. Log in with the same account at `/login`.
3. Land on `/dashboard`, which only shows once the login succeeds.

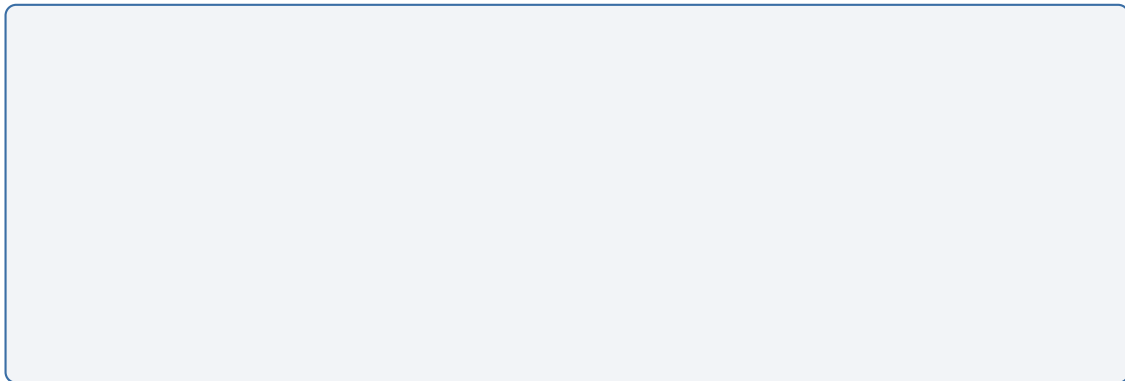


Figure 7: Browser screenshots of the signup page, the login page, and the dashboard after a normal login.

5.2 Dictionary attack, step by step

1. Start the victim webapp, undefended or defended.
2. From the attacker device, run

```
python attacker_dict.py --url http://192.168.68.107:5000 --user alice --wordlist wordlist.txt
```
3. The script sends one `POST /login` request per candidate password in the wordlist, in order, and reads the HTTP status code back.
4. It stops as soon as it sees 302 (password found), 429 (locked out), or the wordlist runs out.

5.3 Known password (spray) attack, step by step

1. Start the victim webapp, undefended or defended.
2. From the attacker device, run
`python attacker_spray.py --url http://192.168.68.107:5000 --userlist targets.txt --passwords known_passwords.txt`
3. The script tries one known password against every account in the target list before moving on to the next known password.
4. It records every attempt and every account where a 302 came back.

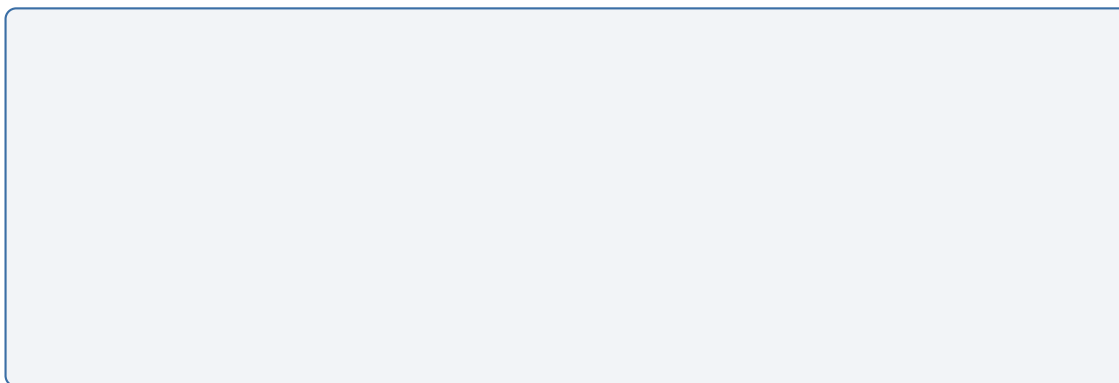


Figure 8: Side by side terminal screenshots: the attacker device running each script, and the victim device's server log scrolling at the same time.

6 Observations

This section is the core of the report. For each attack, in each mode, we describe what we expected from the Design Report, what actually happened according to our own logs from the two-device run, and, where the two do not match exactly, why we think that is. All log excerpts below are copied directly from `src/logs/attacker_dict.log`, `src/logs/attacker_spray.log`, and `src/logs/server.log`, with only the surrounding noise trimmed.

6.1 Dictionary attack, undefended

What we expected. Since `alice`'s password `sunshine` is inside `wordlist.txt`, we expected the attack to find it well before the list ran out, and to do so quickly, since every hash check is a single fast SHA-256.

What we observed. The attack found the password on attempt 49 out of 1000, in about 2 seconds of wall clock time, at roughly 35 to 50 milliseconds per attempt.

```
[ 1.684s] attempt=41 user=alice password='asdfgh' -> HTTP 200
[ 1.732s] attempt=42 user=alice password='hunter' -> HTTP 200
[ 1.755s] attempt=43 user=alice password='buster' -> HTTP 200
[ 1.794s] attempt=44 user=alice password='soccer' -> HTTP 200
[ 1.840s] attempt=45 user=alice password='harley' -> HTTP 200
[ 1.863s] attempt=46 user=alice password='batman' -> HTTP 200
[ 1.905s] attempt=47 user=alice password='andrew' -> HTTP 200
```

```
[ 1.927s] attempt=48 user=alice password='tigger' -> HTTP 200
[ 1.963s] attempt=49 user=alice password='sunshine' -> HTTP 302
CRACKED user=alice password=sunshine attempts=49 time=1.96s
```

Match or not. This matches what we expected. The password was found inside the wordlist, near the top few percent of it, and the attack needed only a couple of seconds of real network traffic to succeed. We reproduced this same result, attempt 49, across several separate runs, which tells us the behaviour is stable and not a one time fluke.

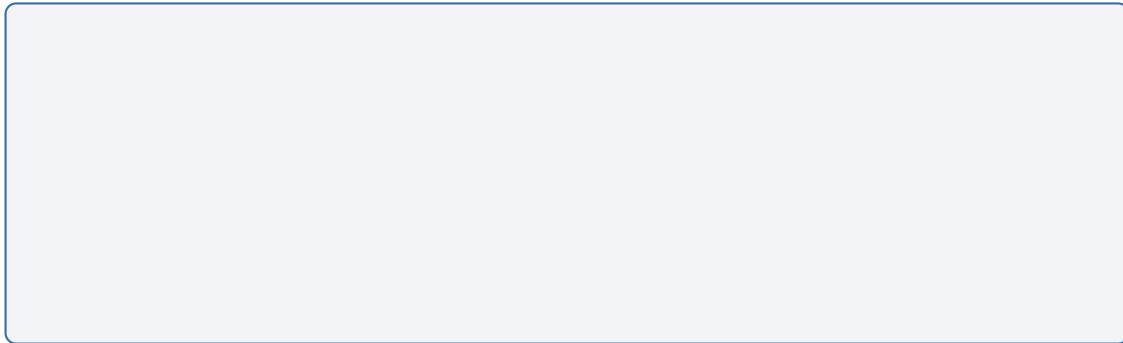


Figure 9: Wireshark capture of the dictionary attack traffic between the attacker and victim devices, Follow HTTP Stream view of one attempt.

6.2 Dictionary attack, defended

What we expected. With the lockout set to 5 failed attempts inside 30 seconds, an attacker who does not already know the password should get locked out after the 5th failure, well before reaching attempt 49. We also expected each attempt to take noticeably longer because of the slower PBKDF2 hash.

What we observed. The account was locked out on attempt 6, before the correct password was ever tried.

```
[ 0.131s] attempt=1 user=alice password='123456' -> HTTP 200
[ 0.252s] attempt=2 user=alice password='password' -> HTTP 200
[ 0.352s] attempt=3 user=alice password='12345678' -> HTTP 200
[ 0.487s] attempt=4 user=alice password='qwerty' -> HTTP 200
[ 0.587s] attempt=5 user=alice password='123456789' -> HTTP 200
[ 0.635s] attempt=6 user=alice password='12345' -> HTTP 429
LOCKED_OUT user=alice attempts=6 time=0.63s
```

Match or not. This matches the design closely. The attack is cut short well before it can reach `sunshine` at position 49, so a dictionary attack that succeeded in the undefended run is completely blocked here. On timing, we measured each attempt taking longer, around 100 to 135 milliseconds per attempt here against roughly 35 to 50 milliseconds in the undefended run, so about two to three times slower. This is a smaller gap than the roughly 200,000 times more hashing work that PBKDF2 at 200,000 iterations actually does per check. The reason is that this measurement is the whole HTTP round trip across the network, which also includes the Flask development server and Python, not the hash function alone. The extra hashing cost is real, but here it is an addition on top of a larger fixed cost. A cleaner way to see the full effect would be timing the hash function directly rather than the full request.

6.3 Known password attack, undefended

What we expected. Since `eve` and `frank` share the password `admin123`, and none of `alice`, `bob`, `carol`, or `dave` use any password from `known_passwords.txt`, we expected the first pass of the spray attack to compromise exactly those two accounts and nothing else.

What we observed. Both accounts were compromised on the very first known password tried, and no other account was ever compromised across the full run of 33 known passwords against 6 accounts.

```
[ 0.042s] user=alice password='admin123' -> HTTP 200
[ 0.088s] user=bob password='admin123' -> HTTP 200
[ 0.134s] user=carol password='admin123' -> HTTP 200
[ 0.158s] user=dave password='admin123' -> HTTP 200
[ 0.182s] user=eve password='admin123' -> HTTP 302
CRACKED user=eve password=admin123
[ 0.206s] user=frank password='admin123' -> HTTP 302
CRACKED user=frank password=admin123
```

Match or not. This matches exactly. The attack needed only one guess per compromised account, which is the whole point of a known password attack: it does not need a large wordlist, it needs the right guess to be shared across accounts.

6.4 Known password attack, defended

What we expected. Based on the Design Report, we expected the logout to throttle this attack per account, the same way it throttles the dictionary attack.

What we observed. `eve` and `frank` were compromised immediately, exactly as in the undefended run. The logout only affected the accounts that were never going to be compromised anyway.

```
[ 0.047s] user=alice password='admin123' -> HTTP 429
[ 0.196s] user=bob password='admin123' -> HTTP 200
[ 0.321s] user=carol password='admin123' -> HTTP 200
[ 0.468s] user=dave password='admin123' -> HTTP 200
[ 0.616s] user=eve password='admin123' -> HTTP 302
CRACKED user=eve password=admin123
[ 0.716s] user=frank password='admin123' -> HTTP 302
CRACKED user=frank password=admin123
[ 0.739s] user=alice password='password1' -> HTTP 429
...
[ 3.420s] user=alice password='Welcome1' -> HTTP 429
[ 3.467s] user=bob password='Welcome1' -> HTTP 429
[ 3.513s] user=carol password='Welcome1' -> HTTP 429
[ 3.561s] user=dave password='Welcome1' -> HTTP 429
```

Match or not, and why it differs. This is the one place where our actual result does not match what we expected in the Design Report, and it is worth explaining clearly. The logout only ever counts a *failed* attempt. For `eve` and `frank`, the very first password we try against them is already the correct one, so the request that compromises them is a success, not a failure, and it happens before the logout counter has any failures to count. The logout only has time to build up against `bob`, `carol`, and `dave`, who fail every single password we try, which is why they end up locked out after enough wrong guesses, even though they were never actually vulnerable to this attack in the

first place. `alice` shows as locked from the very first line here because a dictionary attack against her had already used up her five failures moments earlier in the same test session, so her lockout was still active.

The lesson we take from this is that account lockout, on its own, is a good defense against a dictionary attack, where the attacker has to fail many times before finding the right password, but it does very little against a known password attack aimed at an account that actually reuses a guessed password, because that account is compromised on the first try, before any failure count can matter. Stopping that case would need a different kind of defense, for example blocking known weak or breached passwords at signup time so that no account can ever have `admin123` as its password, or rate limiting logins per source IP address rather than only per username, or requiring a second factor for login regardless of how many failures have happened.

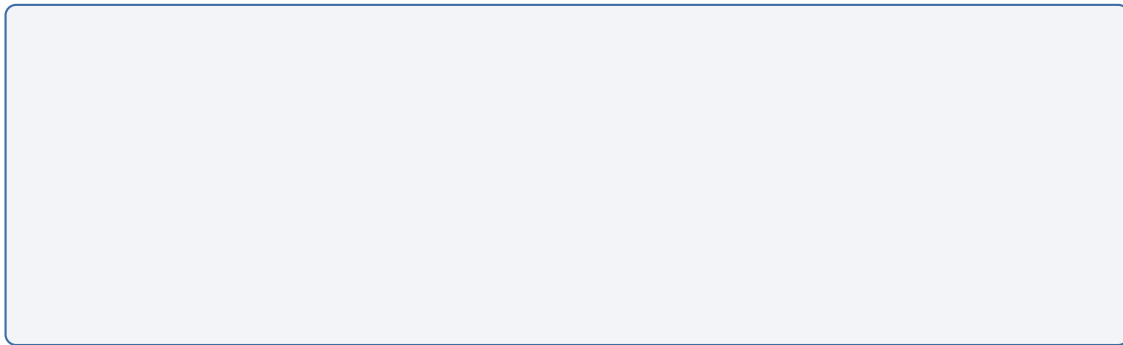


Figure 10: Wireshark capture of the spray attack traffic against the defended server, with the 429 response visible for one of the throttled accounts.

7 Assumptions

Our attacks and our measurements rest on a number of assumptions. We list them here so the results above are read in the right context.

- **The attacker already knows the target usernames.** Both attacker scripts take a username or a list of usernames as input. We do not model how an attacker would first learn valid usernames, for example from a data breach, from an email naming pattern, or from the signup page leaking whether a username is taken.
- **The attacker has direct, unrestricted access to the login endpoint.** There is no firewall, web application firewall, CAPTCHA, or upstream rate limiter between the attacker and the webapp in our setup. A real deployment would likely have at least one of these, which would change the numbers in this report.
- **The attacker sends requests back to back with no delay of its own.** Our scripts do not deliberately slow themselves down or randomize timing to avoid detection. A more careful attacker might spread requests out over a longer time to stay under a lockout window, which our current lockout defense, with its fixed 30 second window, would not fully catch either.
- **The lockout tracker lives in the memory of a single Flask process.** If the victim ran multiple app instances behind a load balancer, each instance would keep its own separate lockout state, and an attacker spread across instances could get more effective guesses than our single

process test shows.

- **The slow hash defense is aimed at a different threat than the one we directly tested.** We tested it through the login endpoint, where the bottleneck is dominated by the web request itself, not by the hash. Its real target is an attacker who has already stolen the password hash table and is guessing offline, with no server and no lockout in the way at all. We did not run that offline scenario here, since it is outside what the login endpoint alone can demonstrate.
- **Timing numbers reflect the local network conditions at the time of testing.** They can vary with Wi-Fi congestion, distance from the access point, or a different network entirely.

8 Defense Mechanism: Design and Assessment

We built two defenses into the webapp, both controlled by one `DEFENSE` flag so that the exact same attacker scripts can be pointed at either configuration.

1. **Account lockout.** The server tracks failed login attempts per username in memory. After 5 failures within a 30 second window, the account is locked for 60 seconds, and every request in that window gets back `429` regardless of the password sent.
2. **Slow password hashing.** In defended mode, passwords are hashed with salted PBKDF2-HMAC-SHA256 at 200,000 iterations instead of a single salted SHA-256 hash.

Based on the observations above, we assess the two parts of our defense separately.

	Expected	Actually observed
Dictionary attack, defended	Blocked by lockout before reaching the correct password	Blocked at attempt 6, matches expectation
Spray attack, defended	Throttled per account	Still succeeds immediately on accounts with a shared weak password, only slows down the accounts that were never actually vulnerable, does not match our original expectation
Cost per login attempt	Higher, on the order of 2×10^5 more hash operations	Measurably higher, roughly two to three times slower end to end, smaller than the theoretical gap because of fixed request overhead

Table 3: Expected versus actually observed effect of the lockout and slow hash defense.

The lockout works exactly as designed against the dictionary attack, and it does add real cost to every login attempt as designed, but it does not fully deliver on what we expected for the spray attack. That gap is not a bug in our implementation, it is a real limitation of what a per account failure counter can ever catch. We think this is a useful finding on its own, since it shows why real systems that care about credential stuffing usually add more than just a lockout, such as breached password blocklists, per IP rate limits, or multi factor authentication.

9 Conclusion

We showed that a dictionary attack and a known password attack can both compromise a realistic signup and login webapp, and that a lockout and slow hashing defense meaningfully changes that outcome, though not in exactly the way we first expected. The dictionary attack cracked a weak password in seconds when undefended and was fully blocked once the lockout was turned on. The known password attack cracked every account that reused a guessed default password, in both the undefended and the defended configuration, which points to a real limitation of a lockout that only counts failures: it cannot stop an attack whose whole point is to succeed on the first try.

The webapp, both attacker scripts, and the defense described in this report are implemented and were tested end to end on two separate devices over a real local network, using the exact commands and log evidence shown above.